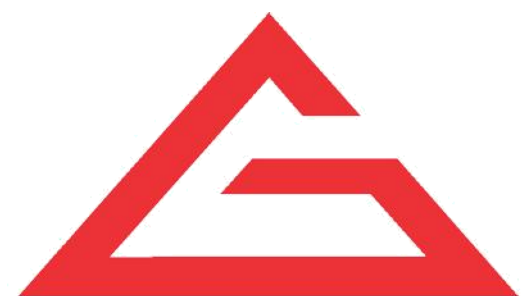




# FLUTTER - 06

## LAB ASSIGNMENTS



**G-TEC EDUCATION**  
— G-TEC Group of Institutions —

[www.gteceducation.com](http://www.gteceducation.com)

Admin Office House of G-TEC, Calicut-02., India. | Corp. Office Peace Centre, Singapore  
— 228149

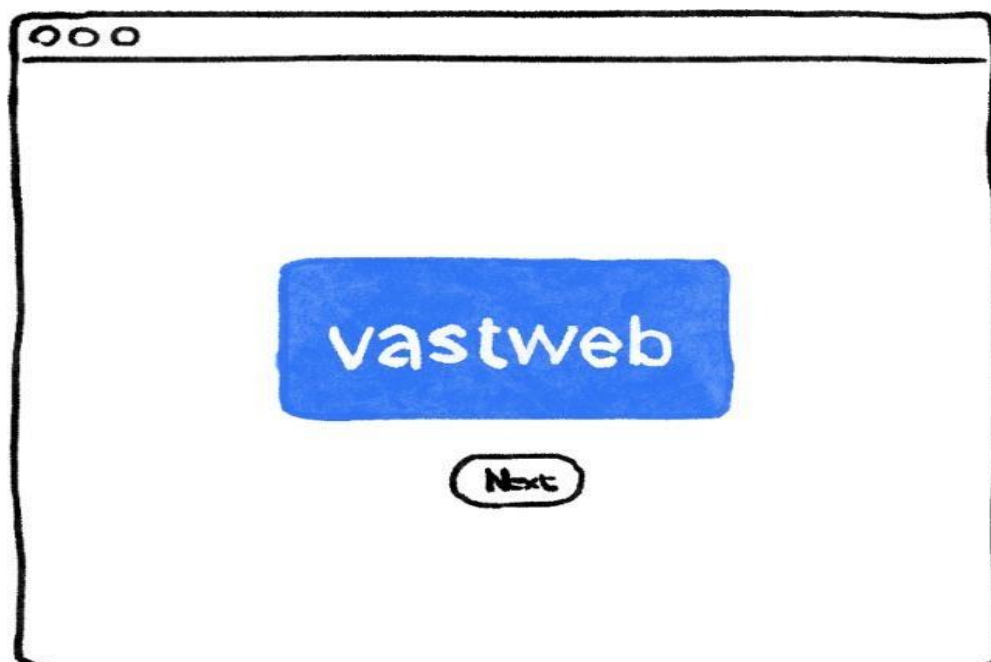
## Make the App Prettier

this is how the app looks at the moment.



Not great. the centerpiece of the app—the randomly generated pair of words—should be more visible. It is, after all, the main reason our users are using this app! Also, the app contents are weirdly off-centre, and the whole app is boringly black & white.

this section addresses these issues by working on the app's design. the end goal for this section is something like the following:



## Extract a widget

the line responsible for showing the current word pair looks like this

now: `Text(appState.current.asLowerCase)`. to change it into something more complex, it's a good idea to extract this line into a separate widget. Having separate widgets for separate logical parts of your UI is an important way of managing complexity in Flutter.

Flutter provides a refactoring helper for extracting widgets but before you use it, make sure that the line being extracted only accesses what it needs. Right now, the line accesses `appState`, but really only needs to know what the current word pair is.

For that reason, rewrite the `MyHomePage` widget as follows:

For that reason, rewrite the `MyHomePage` widget as follows:

[lib/main.dart](#)

```
// ...
class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;          // ← Add this.

    return Scaffold(
      body: Column(
        children: [
          Text('A random AWESOME idea:'),
          Text(pair.asLowerCase),          // ← Change to this.
          ElevatedButton(
            onPressed: () {
              appState.getNext();
            },
            child: Text('Next'),
          ),
        ],
      ),
    );
  }
}
// ...
```

Nice. the `Text` widget no longer refers to the whole `appState`.

Now, call up the **Refactor** menu. In VS Code, you do this in one of two ways:

1. Right click the piece of code you want to refactor (`Text` in this case) and select **Refactor...** from the drop-down menu,

OR

2. Move your cursor to the piece code you want to refactor (`Text`, in this case), and press `Ctrl+`. (Win/Linux) or `Cmd+`. (Mac).

In the **Refactor** menu, select **Extract Widget**. Assign a name, such as **BigCard**, and click `Enter`.

this automatically creates a new class, `BigCard`, at the end of the current file.

the class looks something like the following:

```
lib/main.dart
// ...

class BigCard extends StatelessWidget {
  const BigCard({
    super.key,
    required this.pair,
  });

  final WordPair pair;

  @override
  Widget build(BuildContext context) {
    return Text(pair.asLowerCase);
  }
}

// ...
```

3. Notice how the app keeps working even through this refactoring.

## Add a Card

Now it's time to make this new widget into the bold piece of UI we envisioned at the beginning of this section.

Find the `BigCard` class and the `build()` method within it. As before, call up the **Refactor** menu on the `Text` widget. However, this time you aren't going to extract the widget.

Instead, select **Wrap with Padding**. This creates a new parent widget around the `Text` widget called `Padding`. After saving, you'll see that the random word already has more breathing room.

Increase the padding from the default value of `8.0`. For example, use something like `20` for roomier padding.

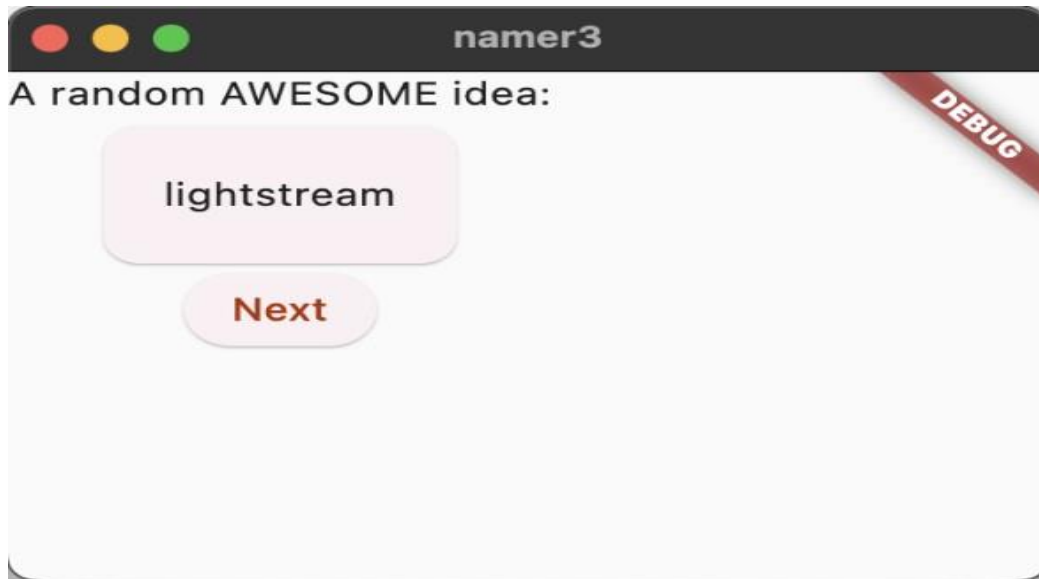
**Note:** Flutter uses Composition over Inheritance whenever it can. Here, instead of padding being an *attribute* of `Text`, it's a widget!

This way, widgets can focus on their single responsibility, and you, the developer, have total freedom in how to compose your UI. For example, you can use the `Padding` widget to pad text, images, buttons, your own custom widgets, or the whole app. The widget doesn't care what it's wrapping.

Next, go one level higher. Place your cursor on the `Padding` widget, pull up the **Refactor** menu, and select **Wrap with widget...**

This allows you to specify the parent widget. Type "Card" and press **Enter**.

This wraps the `Padding` widget, and therefore also the `Text`, with a `Card` widget.



## Theme and style

to make the card stand out more, paint it with a richer color. And because it's always a good idea to keep a consistent color scheme, use the app's `Theme` to choose the color.

Make the following changes to `BigCard`'s `build()` method.

[lib/main.dart](#)

```
// ...
```

```
@override
```

```
Widget build(BuildContext context) {  
  final theme = Theme.of(context); // ← Add this.
```

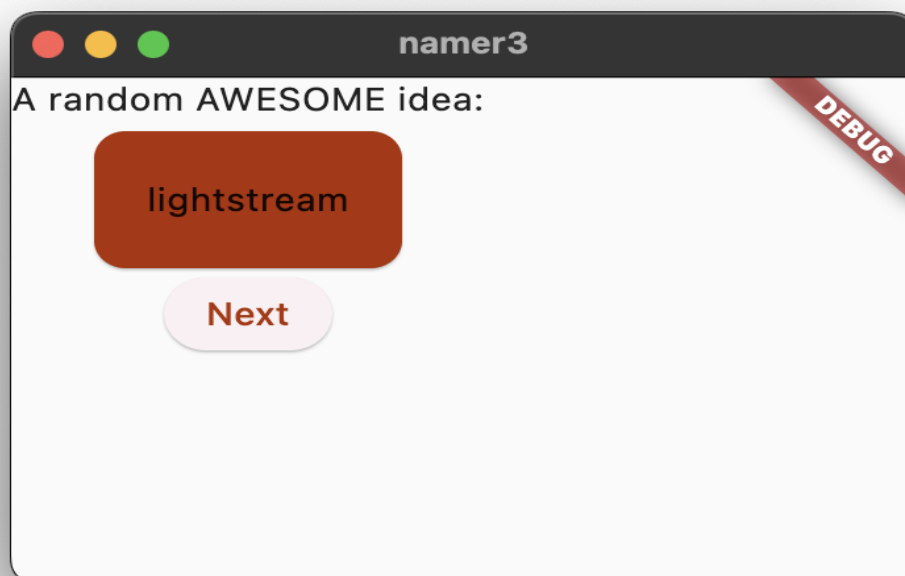
```
  return Card(  
    color: theme.colorScheme.primary, // ← And also this.  
    child: Padding(  
      padding: const EdgeInsets.all(20),  
      child: Text(pair.asLowerCase),  
    ),  
  );  
}
```

```
// ...
```

these two new lines do a lot of work:

- First, the code requests the app's current theme with `Theme.of(context)`.
- then, the code defines the card's color to be the same as the theme's `colorScheme` property. the color scheme contains many colors, and `primary` is the most prominent, defining color of the app.

the card is now painted with the app's primary col



You can change this color, and the color scheme of the whole app, by scrolling up to `MyApp` and changing the seed color for the `ColorScheme` there.

**Tip:** Flutter's `Colors` class gives you convenient access to a palette of hand-picked colors, such as `Colors.deepOrange` or `Colors.red`. But you can, of course, choose any color. To define pure green with full opacity, for example,

use `Color.fromRGBO(0, 255, 0, 1.0)`. If you're a fan of hexadecimal numbers, there's always `Color(0xFF00FF00)`.

Notice how the color animates smoothly. **this** is called an **implicit animation**. Many Flutter widgets will smoothly interpolate between values so that the UI doesn't just "jump" between states.

**the** elevated button below the card also changes color. **that's** the power of using an app-wide **Theme** as opposed to hard-coding values.

## TextTheme

**the** card still has a problem: the text is too small and its color is hard to read. **to** fix this, make the following changes to **BigCard's build()** method.

```
lib/main.dart
// ...

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  // ↓ Add this.
  final style = theme.textTheme.displayMedium!.copyWith(
    color: theme.colorScheme.onPrimary,
  );

  return Card(
    color: theme.colorScheme.primary,
    child: Padding(
      padding: const EdgeInsets.all(20),
      // Change this line.
      child: Text(pair.asLowerCase, style: style),
    ),
  );
}

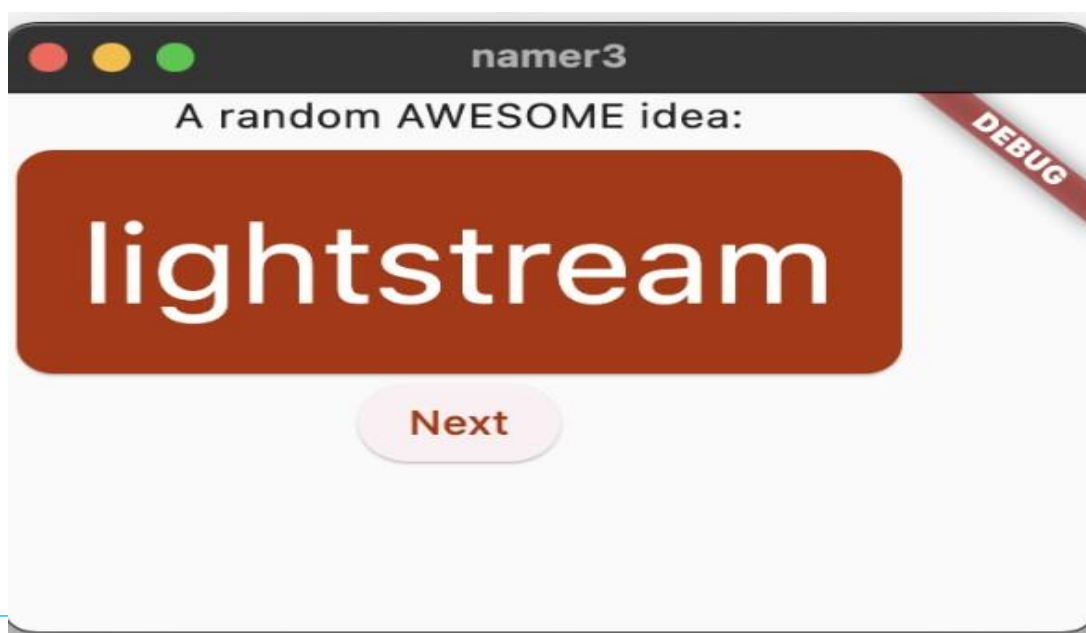
// ...
```

What's behind this change:



- By using `theme.textTheme`, you access the app's font theme. This class includes members such as `bodyMedium` (for standard text of medium size), `caption` (for captions of images), or `headlineLarge` (for large headlines).
- The `displayMedium` property is a large style meant for display text. The word *display* is used in the typographic sense here, such as in [display typeface](#). The documentation for `displayMedium` says that "display styles are reserved for short, important text"—exactly our use case.
- The theme's `displayMedium` property could theoretically be `null`. Dart, the programming language in which you're writing this app, is null-safe, so it won't let you call methods of objects that are potentially `null`. In this case, though, you can use the `!` operator ("bang operator") to assure Dart you know what you're doing. (`displayMedium` is definitely *not* null in this case. The reason we know this is beyond the scope of this codelab, though.)
- Calling `copyWith()` on `displayMedium` returns a *copy* of the text style *with* the changes you define. In this case, you're only changing the text's color.
- To get the new color, you once again access the app's theme. The color scheme's `onPrimary` property defines a color that is a good fit for use *on* the app's *primary* color.

the app should now look something like the following:



If you feel like it, change the card further. Here are some ideas:

- `copyWith()` lets you change a lot more about the text style than just the color. `to` get the full list of properties you can change, put your cursor anywhere inside `copyWith()`'s parentheses, and hit `Ctrl+Shift+Space` (Win/Linux) or `Cmd+Shift+Space` (Mac).
- Similarly, you can change more about the `Card` widget. For example, you can enlarge the card's shadow by increasing the `elevation` parameter's value.
- `try` experimenting with colors. Apart from `theme.colorScheme.primary`, there's also `.secondary`, `.surface`, and a myriad of others. All of these colors have their `onPrimary` equivalents.

## Improve accessibility

Flutter makes apps accessible by default. For example, every Flutter app correctly surfaces all text and interactive elements in the app to screen readers such as `talkBack` and VoiceOver.



Sometimes, though, some work is required. In the case of this app, the screen reader might have problems pronouncing some generated word pairs. While humans don't have problems identifying the two words

in **cheaphead**, a screen reader might pronounce the **ph** in the middle of the word as **f**.

A simple solution is to replace `pair.asLowerCase` with `"${pair.first} ${pair.second}"`. the latter uses string interpolation to create a string (such as "cheap head") from the two words contained in `pair`. Using two separate words instead of a compound word makes sure that screen readers identify them appropriately, and provides a better experience to visually impaired users.

However, you might want to keep the visual simplicity of `pair.asLowerCase`. Use `Text`'s `semanticsLabel` property to override the visual content of the text widget with a semantic content that is more appropriate for screen readers:

### [lib/main.dart](#)

```
// ...
@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final style = theme.textTheme.displayMedium!.copyWith(
    color: theme.colorScheme.onPrimary,
  );

  return Card(
    color: theme.colorScheme.primary,
    child: Padding(
      padding: const EdgeInsets.all(20),

      // ↓ Make the following change.
      child: Text(
        pair.asLowerCase,
        style: style,
        semanticsLabel: "${pair.first} ${pair.second}",
      ),
    ),
  );
}
```

```
}; } // ...
```

Now, screen readers correctly pronounce each generated word pair, yet the UI stays the same. try this in action by [using a screen reader on your device](#).

**Tip:** Flutter has a variety of tools for accessibility, including automated tests and the `Semantics` widget. Learn more on Flutter documentation's [Accessibility page](#).

## Center the UI

Now that the random word pair is presented with enough visual flair, it's time to place it in the center of the app's window/screen.

First, remember that `BigCard` is part of a `Column`. By default, columns lump their children to the top, but we can easily override this. Go to `MyHomePage`'s `build()` method, and make the following change:

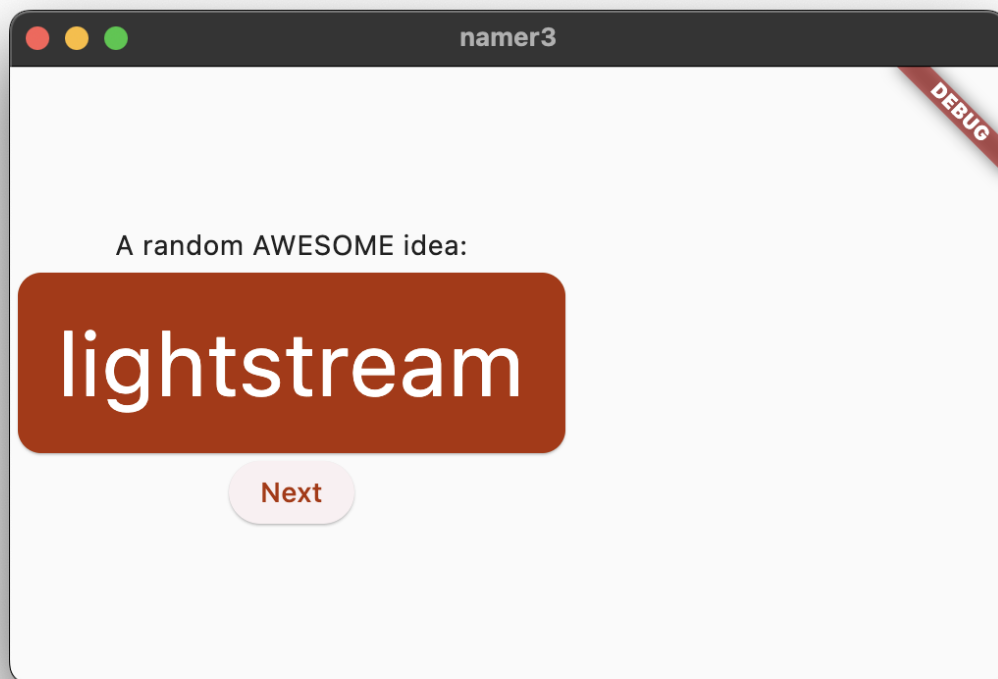
[lib/main.dart](#)

// ...

```
class MyHomePage extends StatelessWidget {  
  @override  
  Widget build(BuildContext context) {  
    var appState = context.watch<MyAppState>();  
    var pair = appState.current;  
  
    return Scaffold(  
      body: Column(  
        mainAxisAlignment: MainAxisAlignment.center, // ← Add this.  
        children: [  
          Text('A random AWESOME idea:'),  
          BigCard(pair: pair),  
          ElevatedButton(  
            onPressed: () {  
              appState.getNext();  
            },  
            child: Text('Next'),  
          ),  
        ],  
      ),  
    );  
  }  
}
```

```
),  
);  
}  
}  
  
// ...
```

this centers the children inside the `Column` along its main (vertical) axis.

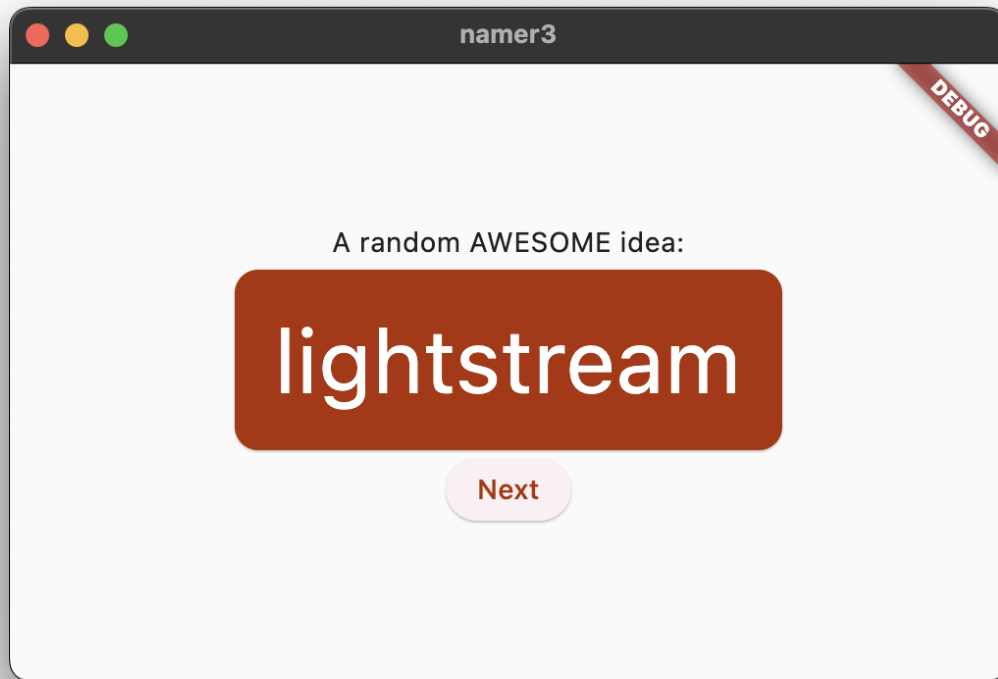


the children are already centered along the column's *cross* axis (in other words, they are already centered horizontally). But the `Column` *itself* isn't centered inside the `Scaffold`. We can verify this by using the **Widget Inspector**.

the Widget Inspector itself is beyond the scope of this codelab, but you can see that when the `Column` is highlighted, it doesn't take up the whole width of the app. It only takes up as much horizontal space as its children need.

You can just center the column itself. Put your cursor onto `Column`, call up the **Refactor** menu (with `Ctrl+.` or `Cmd+.`), and select **Wrap with Center**.

the app should now look something like the following:



If you want, you can tweak this some more.

- You can remove the `Text` widget above `BigCard`. It could be argued that the descriptive text ("A random AWESOME idea:") isn't needed anymore since the UI makes sense even without it. And it's cleaner that way.
- You can also add a `SizedBox(height: 10)` widget between `BigCard` and `ElevatedButton`. This way, there's a bit more separation between the two widgets. The `SizedBox` widget just takes space and doesn't render anything by itself. It's commonly used to create visual "gaps".

With the optional changes, `MyHomePage` contains this code:

[lib/main.dart](#)

// ...

```
class MyHomePage extends StatelessWidget {  
  @override
```



```
Widget build(BuildContext context) {  
  var appState = context.watch<MyAppState>();  
  var pair = appState.current;  
  
  return Scaffold(  
    body: Center(  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          BigCard(pair: pair),  
          SizedBox(height: 10),  
          ElevatedButton(  
            onPressed: () {  
              appState.getNext();  
            },  
            child: Text('Next'),  
          ),  
        ],  
      ),  
    ),  
  );  
}
```

// ...

And the app looks like the following:

